
TreePlugin: Prompting Large Language Models with Trees for Tabular Data

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Few-shot prompting provides an efficient method for adapting Large Language
2 Models (LLMs) to diverse tasks using only a handful of in-context examples.
3 While prior research demonstrates that enriching prompts with structured thought
4 processes can enhance LLMs’ reasoning abilities in areas such as solving math
5 word problems, the potential of few-shot prompting for tabular data prediction
6 remains largely untapped. Traditional methods like chain-of-thoughts do not readily
7 apply in this domain due to the absence of pre-formulated reasoning paths. In
8 this paper, we introduce TreePlugin, a novel approach that incorporates decision
9 trees into query prompts to bridge this gap. We develop a new algorithm to train
10 decision trees based on feedback from LLM predictions, effectively translating
11 LLM insights into actionable, tree-structured guidance. These trees then serve as
12 enhanced prompts that significantly improve LLM performance on tabular data
13 tasks. Our empirical results confirm that TreePlugin not only boosts prediction
14 accuracy but also acts as an interpretable framework to capture and represent the
15 underlying knowledge utilized by LLMs.

16 1 Introduction

17 In recent years, few-shot prompting has emerged as a powerful technique for applying Large Language
18 Models (LLMs) across a broad spectrum of tasks with minimal example input [1–4]. By utilizing
19 in-context learning with a small set of example-output pairs, these models demonstrate remarkable
20 generalization to new, unseen instances. This approach efficiently leverages the extensive pre-trained
21 knowledge embedded within LLMs, enabling them to adapt to specific tasks without extensive
22 retraining, thereby enhancing efficiency and conserving resources. Promising results have been
23 particularly noted in reasoning tasks, such as solving simple math problems [1, 2].

24 Besides reasoning tasks, existing studies [5–9] have also explored the use of LLMs in classifying
25 tabular data, such as disease prediction from health records or anomaly detection in financial transac-
26 tions. Given that tabular data typically requires precision, improving the accuracy of LLM predictions
27 is crucial due to the approximate nature of LLMs. Most research focuses on fine-tuning LLMs for
28 tabular data; however, enhancing prediction from a prompting perspective remains underexplored.

29 A popular method for few-shot prompting involves enriching prompts with explicit reasoning steps [1,
30 4, 3]. For example, Chain-of-Thought (CoT) [1] illustrates the series of reasoning steps that lead to
31 the final answer, aiding the LLM’s reasoning process. Tree-of-Thought (ToT) [3] explores multiple
32 possible reasoning paths from input to answer, using breadth-first search or depth-first search to find
33 the satisfactory path. While these methods excel in tasks with clear, sequential reasoning processes,
34 such as the game of 24, they face limitations in tabular data settings where linking discrete feature
35 values directly to outcomes lacks intuitive textual narratives. This gap underscores the challenge of

36 generating intermediate reasoning steps for tabular data classification, where the progression from
37 input features to a label is not readily verbalizable.

38 Recognizing that decision trees are popular for handling tabular data due to their efficient training
39 and interpretable structure, we introduce TreePlugin, a novel methodology that integrates decision
40 tree logic into the few-shot prompting process of LLMs for tabular data. We dynamically update a
41 decision tree with knowledge extracted from LLM feedback and convert this tree into prompts as
42 guidelines for in-context demonstrations. Our experiments on multiple classification tasks, such as
43 disease diagnosis, demonstrate that TreePlugin not only surpasses existing baselines but also provides
44 insights into the decision-making processes of LLMs through interpretable tree structures. Our main
45 contributions are summarized as follows.

- 46 • We propose a new serialization template to convert tabular data into text, which performs
47 better with a lower number of tokens than previous templates.
- 48 • With extensive experimental studies, we confirm that LLMs possess substantial background
49 knowledge suitable for various tasks and demonstrate that structured tree logic can enhance
50 LLM prompting.
- 51 • Based on the above findings, we propose TreePlugin, a novel approach that enhances
52 LLM few-shot prompting by integrating decision trees extracted from LLM’s background
53 knowledge, thus encapsulating essential reasoning steps for effective prediction.
- 54 • Through comprehensive experiments on multiple applications, we validate that our approach
55 outperforms the other prompting approaches, and we illustrate that the generated trees can
56 serve as explanatory criteria to elucidate the decision-making behavior of LLMs.

57 **2 Background and Related Work**

58 **2.1 Few-Shot Prompting**

59 Few-shot prompting is an efficient way to enhance the ability of LLMs with a limited number of in-
60 context examples. There are many methods that improve few-shot prompting by prompt engineering.
61 Chain-of-Thoughts (CoT) [1] introduces an innovative method of incorporating a “chain of thoughts”
62 to bridge the input and answer, rather than feeding them directly to LLMs. These chains represent
63 meaningful intermediate steps that can bolster the reasoning abilities of LLMs. Skeleton-of-Thought
64 (SoT) [4] firsts guide the LLM to generate the skeleton of the answer and then complete each skeleton
65 point in parallel. Building upon CoT, Tree-of-Thoughts (ToT) [3], facilitates multiple reasoning paths
66 over thoughts, thereby generating more refined results. These approaches cannot be directly applied
67 to the classification of tabular data since the reasoning steps are not available.

68 Another line of work to optimize the prompts is by prompt tuning [10–15]. The basic idea of prompt
69 tuning is to optimize the prefix prompts while freezing the LLMs. For example, [13] optimizes the
70 log-likelihood objective on the input data given the trainable prefix matrix and the frozen language
71 model. Lester et al. [10] prepend the input query with special tokens and tune the embeddings of these
72 tokens directly, which requires fewer parameters to tune. Although prompt tuning does not require
73 updating the LLM, they still need access to the output likelihood and the intermediate embedding
74 layers to tune and prepend the embedding of prompts. Thus, they are not applicable in the black-box
75 LLM setting such as ChatGPT [16] and Claude [17]. In this paper, to increase the usability and
76 efficiency, we focus on the black-box LLM setting where users can only access the input and output
77 of LLMs without other intermediate information.

78 **2.2 LLMs for Tabular Data**

79 Tabular data is a widely used data format and can be easily converted into text by describing the
80 list of feature values. There have been some studies to exploit LLMs for the prediction on tabular
81 data. Existing research [18–20, 5–9] primarily focuses on training or tuning strategies to enhance
82 LLM performance on tabular data. For example, TabLLM demonstrates that fine-tuning the T0
83 model [21] on a limited set of tabular examples can, in certain scenarios, outperform tree-based
84 methods. In this paper, rather than developing better LLMs specifically for tabular data, we aim to
85 uncover the reasoning processes and improve predictions made by LLMs on tabular data through
86 few-shot prompting.

(1) The age is 48. The heart rate is 88. The blood pressure is 98. The evaluation is: low risk (2) The age is 45. The heart rate is 70. The blood pressure is 101. The evaluation is: high risk	(1) age: 48, heart rate: 88, blood pressure: 98, evaluation: low risk (2) age: 45, heart rate: 70, blood pressure: 101, evaluation: high risk	<age>, <heart rate>, <blood pressure>, <RESULT> (1) 48, 88, 98, low (2) 45, 70, 101, high
Text	List	Table

Figure 1: Three templates to convert tabular data into texts.

2.3 Decision Trees

Decision Trees [22–25] are classic and fundamental models used in machine learning and data mining. The core concept behind decision trees is to represent rules in a tree-like structure, where internal nodes denote tests on an attribute, branches represent the outcome of the test, and leaf nodes hold the predictions. The construction of a decision tree involves recursively partitioning the dataset into subsets based on the attribute that results in the maximum information gain or the smallest impurity, such as Gini impurity [22]. Decision trees are easy to understand, interpret, and visualize, which makes them highly attractive for applications where interpretability is crucial.

3 Motivation

3.1 Problem Statement

In this paper, we study the application of few-shot prompting with LLMs for predicting labels from tabular data. Given the in-context examples (x_d, y_d) and the test examples (x_t, y_t) , we aim to design additional prompts z to improve the accuracy of the LLM prediction, i.e., $\arg \min_z \ell(f_\theta(x_t; x_d, y_d, z), y_t)$, where θ is the parameter of the LLM, f is the prediction function, and ℓ is the loss function. Realizing that access to model parameters or intermediate outputs (e.g., embeddings) is usually not allowed when using API services provided by companies such as OpenAI, TogetherAI, or Anthropic, we focus on the prevalent scenarios involving **black-box LLMs**. Prompt tuning approaches [10–15] are infeasible in such scenarios since the embeddings are not accessible.

3.2 Observation 1: Existence of Background Knowledge in LLMs for Tabular Data

Our study begins with an examination of LLMs’ performance in the classification of tabular data.

Table to Text We explore three distinct templates for converting tabular data into text, as illustrated in Figure 1: the text template, the list template, and the tabular template. (1) Text Template: The tabular data is articulated as “The *{feature name}* is *{feature value}*”. (2) List Template: The tabular data is represented as a list of “*{feature name}*: *{feature value}*”. (3) Table Template: The feature names are initially introduced, followed by the presentation of the feature values. The text and list templates, introduced in TabLLM [5], have been demonstrated to outperform other table serialization approaches such as those employing LLMs for text generation. The table template is a new approach proposed by us. We evaluate the three templates in Figure 3a utilizing GPT-3.5 in a few-shot classification setting (#shots=16) on a *diabetes* tabular dataset. The results indicate that our proposed method, the table template, yields the most favorable outcome. Furthermore, the table template requires fewer tokens compared to the other two templates, as it eliminates the need for repetitive feature name articulation, thereby reducing computational costs.

Experimental Study We assess the predictive performance of LLMs on four public tabular datasets under two scenarios: 1) with metadata: besides converting the tabular data into text, we also provide the task information in the query prompt; 2) without metadata: we replace the feature names with IDs and do not provide the task information. The results are presented in Table 1 using zero-shot prompting. From the results, we can observe that the prediction accuracy of LLMs degrades a lot after removing the information about the dataset. The AUC is close to or even lower than 0.5 in many cases, indicating that the LLM’s data mining ability with mere feature IDs is insufficient for accurate prediction on these tabular datasets. The results verify that LLMs indeed contain intrinsic background knowledge about the tasks, and utilizing this understanding of feature names and the prediction tasks is critical for their predictions on tabular data.

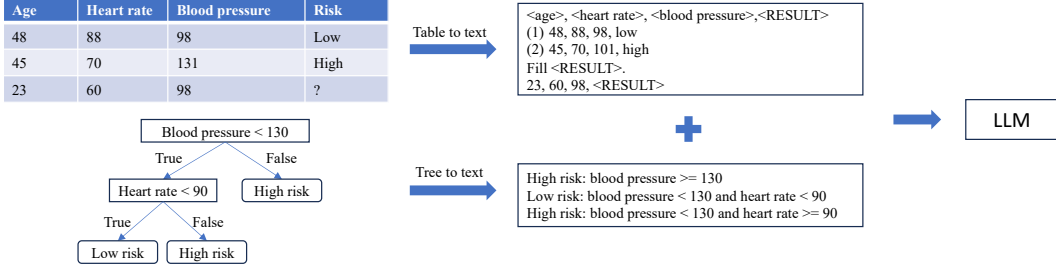


Figure 2: An example of appending trees to the prompts. Besides the original query, we convert the tree into text by describing the split rules and feeding them into the LLM.

Table 1: The test AUC of LLMs with or without the metadata about the dataset. We run three trials and report the mean and standard deviation.

Datasets	GPT-3.5		Llama-2-70b		Vicuna-13b		Mistral-7b	
	w metadata	w/o metadata	w metadata	w/o metadata	w metadata	w/o metadata	w metadata	w/o metadata
diabetes	0.68 ±0.0	0.53±0.0	0.51 ±0.0	0.51±0.0	0.61 ±0.07	0.42±0.0	0.65 ±0.0	0.46±0.0
car	0.71 ±0.0	0.47±0.0	0.69 ±0.0	0.47±0.0	0.87 ±0.01	0.53±0.0	0.8 ±0.0	0.53±0.0
blood	0.52 ±0.01	0.41±0.0	0.66 ±0.0	0.6±0.0	0.68 ±0.0	0.36±0.0	0.69 ±0.0	0.35±0.0
maternal	0.84 ±0.0	0.48±0.0	0.77 ±0.0	0.62±0.0	0.69 ±0.05	0.48±0.0	0.78 ±0.0	0.47±0.0

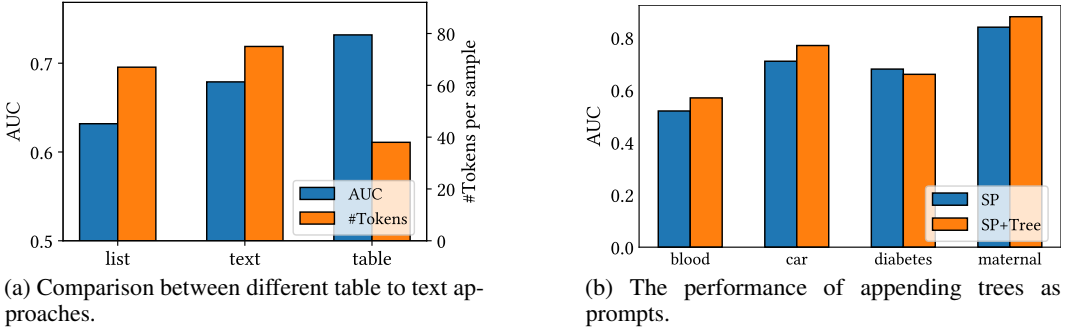


Figure 3: Few-shot classification of GPT-3.5 on *diabetes*.

3.3 Observation 2: Trees can improve Prompting

Existing studies [1, 3, 4] demonstrate that providing structured thoughts can improve prompting. This raises the question: Can structured models, such as decision trees, similarly enhance prompting? To answer this question, we first train a CART tree [22] using the full training dataset. Then we convert the tree into text and add it to the prompts.

Tree to Text Converting a tree to text is a rather intuitive process. As depicted in Figure 2, each path from the root node to a leaf node is described using text in the format “{label}: {split rule}”. These formulated rules are then provided to the LLM to assist in enhancing its predictive capabilities.

Prompting with Trees We compare the following approaches: 1) SP (Standard Prompting): we directly use GPT-3.5 to make predictions on the test dataset with standard few-shot prompting; 2) SP+Tree (Standard Prompting with Tree): we pre-train a CART [22] tree on the full training dataset (the training dataset is much larger than the in-context examples). Then, we convert the CART tree into the prompt and feed it into the LLM together with the in-context examples, and then use the LLM to predict the test dataset. Results are illustrated in Figure 3b. Among four datasets, SP+Tree can outperform SP in most cases, indicating that LLM can successfully understand high-quality prompts derived from decision trees. However, in the few-shot setting when we only have a few in-context examples, we cannot train a valid decision tree.

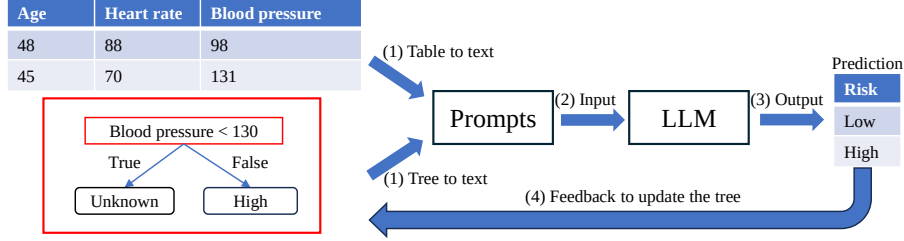


Figure 4: The general pipeline of updating the decision tree from the feedback of the LLM. There are four steps in each round: 1) Convert the tabular data and the current tree into prompts; 2) Input the prompts to the LLM; 3) Compute the loss based on the prediction of the LLM; 4) Update a tree node with the minimal loss.

4 Method

Based on the above observations, we propose TreePlugin, where we **first extract the knowledge of the LLM into a tree, and then use the tree as the explicit reasoning steps to assist the prediction of LLMs**. Next, we introduce the structure of the decision tree and the framework to train the tree.

4.1 Decision Tree with Unknown Leaves

In traditional decision tree models, each leaf node houses a prediction value, ensuring that every potential input instance receives a prediction. However, in our methodology, where decision trees are transformed into text, assigning a definite prediction value to every leaf node is not necessary, especially for nodes without high confidence. To accommodate instances where the tree is unable to render confident predictions based on the split rules, we introduce an “unknown” option. During the conversion of the tree into text, only the paths from the root node to the leaf nodes that are not designated as “unknown” are serialized. The incorporation of unknown leaves allows us to filter out noisy rules and retain only those that are valuable for both prediction and explanation.

4.2 Greedy Tree Training with LLM Feedback

As illustrated in Figure 4, our method involves utilizing the feedback generated by the LLM to train the decision tree. Given the substantial computational cost associated with traversing all possible tree structures to identify the optimal one, we adopt a greedy strategy to update the tree, progressing from the top to the bottom. At each node, we explore all potential split values to pinpoint the one that yields the minimal loss, designating it as the optimal node. Specifically, during the update of a node, we consider all possible leaf values as the child nodes of the current node. In the context of binary classification, the leaf value can be 0, 1, or “unknown”. Subsequently, for each tree structure generated, we convert it into prompts and append them to the original query prompt, thereby soliciting predictions from the LLM. The loss is calculated based on the L_1 norm distance between the predicted values and the actual labels, expressed as $\ell_{LLM} = \frac{1}{|\mathcal{D}|} \sum_i |\mathbf{y}_i^p - \mathbf{y}_i|$, where $\mathcal{D}$ represents the training data, $\mathbf{y}^p$ denotes the values predicted by the LLM, and $\mathbf{y}$ signifies the actual labels.

To maintain the interpretability of the tree and avoid the potential misguidance of the LLM by the tree rules, we incorporate a regularization term into the objective. This term evaluates the predictions made solely by the tree. Given that the predictions from tree leaves might be unknown, we introduce a hyperparameter μ to represent the loss associated with unknown predictions. Suppose $\mathbf{y}^{pt}$ is the prediction of the training data from the tree, the loss attributed to the tree is

$$\ell_{tree} = \frac{1}{|\mathcal{D}|} \sum_i \begin{cases} |\mathbf{y}_i^{pt} - \mathbf{y}_i| & \text{if } \mathbf{y}_i^{pt} \text{ is not unknown} \\ \mu & \text{if } \mathbf{y}_i^{pt} \text{ is unknown} \end{cases}. \quad (1)$$

Then, our final loss to evaluate a tree structure is $\ell = \ell_{LLM} + \lambda \ell_{tree}$, where λ is a hyperparameter to control the weight of the regularization term.

The comprehensive algorithm is presented in Algorithm 1. For clarity in presentation, we employ $\mathbf{T}$ to denote the tree model, wherein $\mathbf{T}[i]$ signifies the i -th tree node of $\mathbf{T}$ in a top-to-bottom sequence

Algorithm 1: The TreePlugin approach.

Input: Depth of tree E . Training data $\mathcal{D} = \{\mathbf{x}, \mathbf{y}\}$

Output: The final tree model $\mathbf{T}$

```

1 QueryPrompts  $\leftarrow$  Table2Text( $\mathcal{D}$ ) // Generate query prompts
2  $\mathbf{S} \leftarrow$  ProposeSplitPoints( $\mathbf{x}$ ) // Generate split candidates
3 /* Update tree node one by one */
4 for  $i = 1, \dots, 2^E - 1$  do
5      $\ell_{min} \leftarrow \infty$ 
6     /* Traverse all possible split candidates */
7     for  $s$  in  $\mathbf{S}$  do
8          $\mathbf{T}' \leftarrow \mathbf{T}$ 
9          $\mathbf{T}'[i] \leftarrow s$  // set current node to candidate value
10        /* Traverse all possible leaf values */
11        for (LeftValue, RightValue) in  $\{0, 1, \text{unknown}\} \times \{0, 1, \text{unknown}\}$  do
12             $\mathbf{T}'[2i] \leftarrow \text{LeftValue}$  // set the left child of node  $i$ 
13             $\mathbf{T}'[2i + 1] \leftarrow \text{RightValue}$  // set the right child of node  $i$ 
14            TreePrompts  $\leftarrow$  Tree2Text( $\mathbf{T}'$ )
15             $\mathbf{y}^p \leftarrow \text{LLM}(\text{QueryPrompts}, \text{TreePrompts})$ 
16             $\mathbf{y}^{pt} \leftarrow \mathbf{T}'(\mathbf{x})$ 
17             $\ell \leftarrow \ell_{LLM}(\mathbf{y}^p, \mathbf{y}) + \lambda \ell_{tree}(\mathbf{y}^{pt}, \mathbf{y})$ 
18            /* Update the node if the loss is minimal */
19            if  $\ell < \ell_{min}$  then
20                 $\ell_{min} \leftarrow \ell$ 
21                 $\mathbf{T} \leftarrow \mathbf{T}'$ 
22            if  $\ell == 0$  then
23                return  $\mathbf{T}$  // early stopping
24 return  $\mathbf{T}$ 

```

(for instance, $\mathbf{T}[1]$ is the root node). Initially, we generate the query prompts from the training data utilizing the table template. Additionally, we formulate candidate split values derived from the feature values, which are generated based on histograms. Specifically, these candidate split values uniformly divide the training data into distinct bins [26] (Line 2). For each tree node, we explore all feasible split values and leaf values and compute the loss according to (1) (Line 3-17). If the calculated loss is found to be smaller than the current minimal loss, the tree is updated with the current nodes (Line 18-21). The tree undergoes updates until it attains the maximum depth or until the loss reaches a minimal state (Line 22-24). After generating the tree, we convert the tree into prompts along with the in-context examples for prompting as presented in Figure 2.

5 Evaluation

5.1 Experimental Setup

Datasets and Tasks We conduct experiments on four public tabular datasets from UCI machine learning repository [27] covering tasks from disease prediction to age prediction: *diabetes* [28], *blood* [29], *car* [30], and *maternal* [31]. For each dataset, we randomly sample 100 examples as the test dataset and then sample a few instances from the remained data as the training/in-context examples for few-shot prompting. For the detailed statistics of each dataset, please refer to Appendix B.

Approaches We compare TreePlugin with the following prompting approaches: 1) **SP** (Standard Prompting): we directly feed the training data to the LLM and then ask it to make prediction on the test data; 2) **SP-Tree** (Standard Prompting with Tree): We train a tree using the training examples and add the tree into the standard prompts using the tree to text. 3) **CoT-LLM** (Chain-of-Thoughts with LLMs for thought generation) [1]: we first directly ask LLMs to generate the reasoning paths for the training samples, and then provide the reasoning paths with the training examples in few-shot

Table 2: The AUC of different prompting approaches in the few-shot classification setting. We run five trials for each experiment and report the mean AUC and std.

Datasets	#Shots	SP	SP-Tree	CoT-LLM	CoT-Tree	TreePlugin
diabetes	2	0.71 $\pm$ 0.06	0.64 $\pm$ 0.07	0.67 $\pm$ 0.06	0.49 $\pm$ 0.11	0.73$\pm$0.02
	4	0.68 $\pm$ 0.04	0.50 $\pm$ 0.07	0.67 $\pm$ 0.06	0.61 $\pm$ 0.08	0.72$\pm$0.03
	8	0.67 $\pm$ 0.05	0.68 $\pm$ 0.04	0.69 $\pm$ 0.04	0.64 $\pm$ 0.06	0.70$\pm$0.06
car	2	0.78 $\pm$ 0.07	0.81$\pm$0.08	0.80 $\pm$ 0.04	0.64 $\pm$ 0.11	0.78 $\pm$ 0.04
	4	0.78 $\pm$ 0.05	0.77 $\pm$ 0.06	0.79 $\pm$ 0.04	0.74 $\pm$ 0.03	0.80$\pm$0.40
	8	0.89 $\pm$ 0.03	0.86 $\pm$ 0.06	0.85 $\pm$ 0.03	0.81 $\pm$ 0.03	0.91$\pm$0.01
blood	2	0.68 $\pm$ 0.01	0.69 $\pm$ 0.02	0.63 $\pm$ 0.02	0.70$\pm$0.04	0.70$\pm$0.04
	4	0.62 $\pm$ 0.09	0.60 $\pm$ 0.09	0.63 $\pm$ 0.09	0.63$\pm$0.09	0.62 $\pm$ 0.07
	8	0.65 $\pm$ 0.05	0.65 $\pm$ 0.07	0.61 $\pm$ 0.03	0.64 $\pm$ 0.04	0.65$\pm$0.07
maternal	2	0.81 $\pm$ 0.01	0.76 $\pm$ 0.03	0.84 $\pm$ 0.05	0.72 $\pm$ 0.07	0.86$\pm$0.02
	4	0.84 $\pm$ 0.05	0.83 $\pm$ 0.04	0.80 $\pm$ 0.02	0.81 $\pm$ 0.03	0.86$\pm$0.03
	8	0.88 $\pm$ 0.02	0.86 $\pm$ 0.02	0.84 $\pm$ 0.02	0.81 $\pm$ 0.1	0.89$\pm$0.02

prompting; 4) **CoT-Tree** (Chain-of-Thoughts with Trees for thought generation): we first use the training examples to train a CART, and then use the CART as the guideline to provide the reasoning path for each training example for CoT. We do not compare with ToT [3] since it requires a state evaluator which is not available in our setting. For TreePlugin, we tune $\lambda \in \{0, 0.1, 1, 2, 4\}$ and $\mu \in \{0, 0.2, 0.6, 1, 2\}$. We also try removing the unknown leaves in the experiments to cover the case when μ is extremely large. We also compare the tree generated by TreePlugin (named **TPT**) with the **CART** [22] tree that is directly trained on the in-context examples.

Prompts Our prompting template has the following components: system instruction, dataset information, data format description, in-context tabular data description by table to text, tree description by tree to text, prediction instructions, and test data description by table to text. The tree description part is only used in TreePlugin and SP-Tree. The other parts are used in all prompting approaches. For the detailed prompts, please refer to Appendix A.

Models We use GPT-3.5 (gpt-3.5-turbo-0613) as the main model in the experiments. We set the GPT *temperature* parameter to 0 to improve its stability and reproducibility given the same input. For the experiments on other LLMs, please refer to Appendix D.3. For the decision tree, considering that the number of features is less than 10, we set the maximum number of depth to 3 for all approaches by default to ensure a fair comparison. The number of bins is set to 10.

Setup We run the experiments on a Linux server with 4x AMD EPYC 7543 32-Core Processor. By default, we run five trials for each experiment and report the mean and standard derivation.

5.2 Comparison among Different Prompting Methods

We evaluate the efficacy of our proposed methods against baseline approaches in a few-shot classification task, varying the number of training instances from 2 to 8. The results are detailed in Table 2. We observe that 1) TreePlugin outperforms other prompting methods, securing the best results in a majority of scenarios and underscoring the effectiveness of our approach. In instances where baseline methods achieve superior performance, TreePlugin still remains closely competitive, registering near-optimal accuracy. 2) CoT-LLM and CoT-Tree even perform worse than SP in many cases, indicating that directly using the LLM or tree trained on the in-context examples to generate reasoning paths for tabular data is not effective. Moreover, CoT-Tree usually have a worse performance than CoT-LLM, indicating that a good tree is necessary to improve prompting. 3) Increasing the number of in-context examples does not necessarily improve the performance of the prompting methods. The quality of

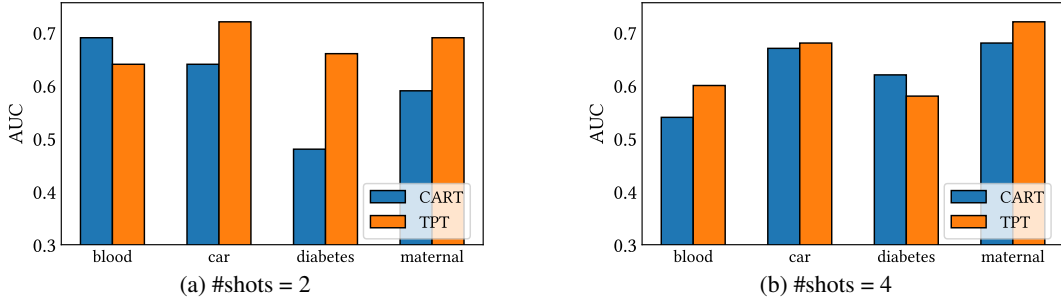


Figure 5: Comparison between different tree approaches.

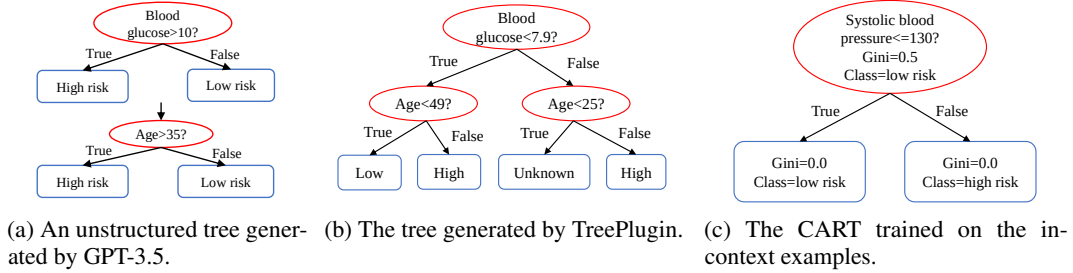


Figure 6: Comparison of interpretability of different tree models.

in-context examples could be more important than the quantity. If the additional examples are not diverse or representative enough of the test data, they might not provide the necessary variability to guide the LLM towards better generalization.

5.3 Comparison among Different Trees

To assess the knowledge extraction performance of TreePlugin, we compare the tree generated by our method with the other trees as presented in Figure 5. From the results, we observe that the tree generated by TreePlugin has better performance than the other trees in most cases. While CART is trained solely on limited training instances, our tree absorbs knowledge from both the training data and the LLM. This fusion results in the formulation of more effective rules for classification tasks.

The tree generated by TreePlugin also serves as an interpretability tool for the prediction of LLMs. We have also visualized our tree, the tree produced by directly asking LLM to generate one (prompt: please summarize the rules used for the prediction in a binary tree), and the CART tree trained on the in-context examples in Figure 6a. The LLM falls short in generating a single structured binary tree; instead, it merely links the trees representing various factors, neglecting to delineate the relationships between them. In contrast, as depicted in Figure 6b, our tree effectively visualizes the task rules and is readily interpretable compared to the explanation offered by the LLM. Additionally, when juxtaposed with the CART shown in Figure 6c, our tree emerges as more rational. The CART model is susceptible to overfitting, simply labeling the instances as low risk given a normal feature value. Our tree introduces the “unknown” value for instances with scarce normal feature values, thereby allowing the LLM to base predictions on additional features not encompassed by the tree.

5.4 Multiple Trees

In Algorithm 1, we delineate the process of training a single tree. However, our methodology can also be seamlessly extended to accommodate the training of multiple trees through feature bagging. Specifically, we partition the feature space into several random subsets and independently train a tree for each. Subsequently, the prompts generated by all these trees are fed into the LLM. The outcomes, utilizing three trees (#shots=8), are showcased in Table 3. Evidently, our strategy surpasses both the standalone LLM and the random forest baselines in all four datasets, underscoring the scalability and adaptability of our approach.

Table 3: Comparison between different approaches with multiple trees. RF is a random forest approach provided by scikit.

	SP	RF	TreePlugin
diabetes	0.67 $\pm$ 0.05	0.56 $\pm$ 0.04	0.70 $\pm$ 0.05
car	0.89 $\pm$ 0.03	0.71 $\pm$ 0.12	0.93 $\pm$ 0.02
blood	0.65 $\pm$ 0.05	0.65 $\pm$ 0.07	0.66 $\pm$ 0.07
maternal	0.69 $\pm$ 0.07	0.88 $\pm$ 0.02	0.84 $\pm$ 0.03

Table 4: Ablation study by removing the unknown option in leaf values or the regularization term.

	TreePlugin	w/o unknown	w/o ℓ_{tree}
diabetes	0.70 $\pm$ 0.06	0.69 $\pm$ 0.05	0.67 $\pm$ 0.06
car	0.93 $\pm$ 0.01	0.88 $\pm$ 0.01	0.83 $\pm$ 0.08
blood	0.66 $\pm$ 0.07	0.61 $\pm$ 0.06	0.64 $\pm$ 0.06
maternal	0.89 $\pm$ 0.02	0.86 $\pm$ 0.02	0.89 $\pm$ 0.02

5.5 Ablation Study

We explore the impact of incorporating unknown leaves and the regularization term, as depicted in (1), within our methodology. The corresponding results are showcased in Table 4, wherein the number of training instances is fixed at eight. The outcomes indicate that our approach derives substantial benefits from the introduction of both the unknown option and the tree-based regularization term. The improvement of these techniques is about 2-3% AUC on average.

5.6 Sensitivity Study

Our approach introduces hyperparameters λ and μ . Table 5 presents the model performance of our approach with different hyperparameters on diabetes with eight in-context examples. From the results, we can observe that using $\lambda \in [1, 2]$ and $\mu \in [0.01, 0.2]$ can achieve satisfactory accuracy. Our approach is stable in a reasonable range of hyperparameters.

Table 5: Sensitivity of TreePlugin on different hyperparameters λ and μ on diabetes.

	$\mu=0$	$\mu=0.01$	$\mu=0.2$
$\lambda=0.1$	0.65 $\pm$ 0.05	0.65 $\pm$ 0.05	0.65 $\pm$ 0.04
$\lambda=1$	0.66 $\pm$ 0.04	0.66 $\pm$ 0.05	0.66 $\pm$ 0.04
$\lambda=2$	0.66 $\pm$ 0.04	0.67 $\pm$ 0.04	0.66 $\pm$ 0.04

6 Discussion and Limitations

Societal Impacts The work presented in this paper on enhancing few-shot prompting with TreePlugin holds several potential societal impacts. The ability to efficiently predict outcomes from tabular data using LLMs, particularly in settings with limited data, could significantly benefit fields such as healthcare and finance where quick, accurate decisions are crucial. For example, a family can use our method to predict disease where the in-context examples are from the family members’ health records. Moreover, the interpretability provided by TreePlugin could increase transparency in AI decision-making, fostering trust among users and compliance with regulatory standards.

Limitations Our method exhibits certain limitations: First, while the local computation overhead of TreePlugin is low, it necessitates the training of a decision tree through multiple queries to the LLM, which incurs higher inference costs compared to directly employing an LLM for thought generation. Assume that the number of features is d and the depth of tree is E , TreePlugin requires $O(d \cdot 2^E)$ queries when training the tree. Details on the overhead associated with TreePlugin and strategies to mitigate these costs are discussed in Appendix D.4. Second, TreePlugin is tailored for few-shot settings and may not provide additional benefits in rich-data scenarios where directly training a decision tree on ample examples could suffice.

7 Conclusion

This study proposes TreePlugin, a novel methodology that enhances few-shot prompting in LLMs by integrating decision trees for tabular data prediction. Our results demonstrate that TreePlugin not only boosts the predictive accuracy of LLMs but also provides interpretable insights into their decision-making processes. Moreover, TreePlugin demonstrates that it is promising to introduce other lightweight machine learning models as plugins in prompts.

References

- [1] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.
- [2] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- [3] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *arXiv preprint arXiv:2305.10601*, 2023.
- [4] Xuefei Ning, Zinan Lin, Zixuan Zhou, Zifu Wang, Huazhong Yang, and Yu Wang. Skeleton-of-thought: Prompting LLMs for efficient parallel generation. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=mqVgBbNCm9>.
- [5] Stefan Hegselmann, Alejandro Buendia, Hunter Lang, Monica Agrawal, Xiaoyi Jiang, and David Sontag. Tabllm: Few-shot classification of tabular data with large language models. In *International Conference on Artificial Intelligence and Statistics*, pages 5549–5581. PMLR, 2023.
- [6] Sukriti Jaitly, Tanay Shah, Ashish Shugani, and Razik Singh Grewal. Towards better serialization of tabular data for few-shot classification with large language models, 2023.
- [7] Xi Fang, Weijie Xu, Fiona Anting Tan, Jiani Zhang, Ziqing Hu, Yanjun Qi, Scott Nickleach, Diego Socolinsky, Srinivasan Sengamedu, and Christos Faloutsos. Large language models (llms) on tabular data: Prediction, generation, and understanding-a survey. *arXiv preprint arXiv:2402.17944*, 2024.
- [8] Tuan Dinh, Yuchen Zeng, Ruisu Zhang, Ziqian Lin, Michael Gira, Shashank Rajput, Jy yong Sohn, Dimitris Papailiopoulos, and Kangwook Lee. LIFT: Language-interfaced fine-tuning for non-language machine learning tasks. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022. URL https://openreview.net/forum?id=s_PJMEGIUfa.
- [9] Han Zhang, Xumeng Wen, Shun Zheng, Wei Xu, and Jiang Bian. Towards foundation models for learning on tabular data, 2024. URL <https://openreview.net/forum?id=h2z2haZPXm>.
- [10] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691*, 2021.
- [11] Xiao Liu, Kaixuan Ji, Yicheng Fu, Weng Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. P-tuning: Prompt tuning can be comparable to fine-tuning across scales and tasks. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 61–68, 2022.
- [12] Zexuan Zhong, Dan Friedman, and Danqi Chen. Factual probing is [mask]: Learning vs. learning to recall. *arXiv preprint arXiv:2104.05240*, 2021.
- [13] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190*, 2021.
- [14] Guanghui Qin and Jason Eisner. Learning how to ask: Querying lms with mixtures of soft prompts. *arXiv preprint arXiv:2104.06599*, 2021.
- [15] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, 2023.
- [16] OpenAI. Gpt-4 technical report, 2023.

- [17] Anthropic. Introducing the next generation of claude, March 2024. URL <https://www.anthropic.com/news/claude-3-family>. Accessed: 2024-05-17.
- [18] Zifeng Wang, Chufan Gao, Cao Xiao, and Jimeng Sun. Anypredict: Foundation model for tabular prediction. *arXiv preprint arXiv:2305.12081*, 2023.
- [19] Jonathan Herzig, Paweł Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Martin Eisenschlos. Tapas: Weakly supervised table parsing via pre-training. *arXiv preprint arXiv:2004.02349*, 2020.
- [20] Tianping Zhang, Shaowen Wang, Shuicheng Yan, Jian Li, and Qian Liu. Generative table pre-training empowers models for tabular prediction. *arXiv preprint arXiv:2305.09696*, 2023.
- [21] Victor Sanh, Albert Webson, Colin Raffel, Stephen H Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Teven Le Scao, Arun Raja, et al. Multitask prompted training enables zero-shot task generalization. *arXiv preprint arXiv:2110.08207*, 2021.
- [22] Wei-Yin Loh. Classification and regression trees. *Wiley interdisciplinary reviews: data mining and knowledge discovery*, 1(1):14–23, 2011.
- [23] Oded Z Maimon and Lior Rokach. *Data mining with decision trees: theory and applications*, volume 81. World scientific, 2014.
- [24] J. Ross Quinlan. Induction of decision trees. *Machine learning*, 1:81–106, 1986.
- [25] J Ross Quinlan. *C4. 5: programs for machine learning*. Elsevier, 2014.
- [26] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems*, 30, 2017.
- [27] Arthur Asuncion and David Newman. Uci machine learning repository, 2007.
- [28] Michael Kahn. Diabetes. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5T59G>.
- [29] I-Cheng Yeh. Blood Transfusion Service Center. UCI Machine Learning Repository, 2008. DOI: <https://doi.org/10.24432/C5GS39>.
- [30] Marko Bohanec. Car Evaluation. UCI Machine Learning Repository, 1997. DOI: <https://doi.org/10.24432/C5JP48>.
- [31] Marzia Ahmed. Maternal Health Risk. UCI Machine Learning Repository, 2023. DOI: <https://doi.org/10.24432/C5DP5D>.
- [32] Peter Kairouz, H Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. Advances and open problems in federated learning. *Foundations and Trends® in Machine Learning*, 14(1–2):1–210, 2021.
- [33] Qiang Yang, Yang Liu, Tianjian Chen, and Yongxin Tong. Federated machine learning: Concept and applications. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 10(2):1–19, 2019.
- [34] Qinbin Li, Zeyi Wen, Zhaomin Wu, Sixu Hu, Naibo Wang, Yuan Li, Xu Liu, and Bingsheng He. A survey on federated learning systems: Vision, hype and reality for data privacy and protection. *IEEE Transactions on Knowledge and Data Engineering*, 2021.
- [35] Warwick Nash, Tracy Sellers, Simon Talbot, Andrew Cawthorn, and Wes Ford. Abalone. UCI Machine Learning Repository, 1995. DOI: <https://doi.org/10.24432/C55C7W>.

A Prompting Templates

To make our approach easily generable to different tasks, we design a prompting templates to send user queries as shown in Listing 1. Text in **blue** font denotes the beginning or end of a block, **red** font signifies variables that vary across datasets, **orange** font represents system commands, and text in **black** font is the fixed input text used for prompting.

B Statistics of Datasets

The statistics of datasets used in our experiments are presented in Table 6. Due to the limitation of context length in existing LLMs, we are not able to input very high-dimensional tabular dataset into LLMs.

Table 6: The information of datasets used in our experiments.

Datasets	#instances	#features	Task description
blood	748	4	blood donation prediction
car	1,275	6	car acceptability prediction
diabetes	768	8	disease diagnosis
maternal	679	6	pregnancy risk prediction

C Table to Text

We compare the number of tokens by different serialization approaches in Table 7. The table template requires fewer tokens compared to the other two templates, as it eliminates the need for repetitive feature name articulation, thereby reducing computational costs. Thus, our proposed table template is the best option for strong LLMs like GPT-3.5, while the list template is a good option for moderate LLMs considering both effectiveness and query costs.

D Additional Experimental Results

D.1 Interpretability

Figure 7 shows the example of the prediction and explanation of the LLM. LLMs are knowledgeable, and thus they can make good predictions on the given tabular data in some cases. However, in providing explanations, the LLM typically enumerates the factors contributing to the prediction, exhibiting several limitations: 1) Normal feature values (e.g., age), which typically are not crucial for reasoning, are also reported; 2) The influential factors are delineated for each individual instance, hindering users from readily identifying systematic, quantified key contributors to the task. 3) No knowledge of the relation between multiple factors is provided.

Table 7: The number of tokens per sample of different templates. For list and table templates, we also compute the reduced costs compared with text.

Datasets	#tokens per sample (reduced cost (%))		
	text	list	table
diabetes	75	66 (-12%)	36 (-52%)
car	48	42 (-13%)	22 (-53%)
blood	39	34 (-13%)	19 (-51%)
maternal	61	54 (-11%)	27 (-56%)

```

1 % block system_directive %
2 You are a good data analyst that can find patterns of given data. You
  will be given a list of rules that you must follow, and you should
  predict the output of the given data based on the rules and your
  observation. Please don't give explanations.
3 % endblock %
4
5 % block dataset_intro %
6 Here's your task. The data below is used to [dataset.target]
7
8 The data consists of the following features:
9   for feature in dataset.features
10     [feature.name, feature.description]
11   endfor
12
13 For each data, the result can be one of the following:
14   for label in dataset.labels
15     [label.name, label.description]
16   endfor
17 % endblock %
18
19
20 % block format_descriptions %
21 Here's how the data will be presented. For each line:
22   [features], <RESULT>
23 % endblock %
24
25 % block incontext_examples %
26 These are some known data points:
27   for example in examples
28     [example with optional thoughts]
29   endfor
30 % endblock %
31
32 % block tree_rules %
33 The rules are as follows:
34   for rule in rules
35     [rule]
36   endfor
37 % endblock %
38
39 % block prediction_intro %
40 For data points that are not covered by any rule, you should predict
  the result based on your observation.
41
42 Please make prediction on the following [n_instances] lines containing
  "<RESULT>", filling "<RESULT>" with one of [labels], and reply
  each prediction in a new line. Make sure there are exactly [
  n_instances] lines.
43   for instance in instances
44     [instance.features], <RESULT>
45   endfor
46 % endblock %

```

Listing 1: Prompting Template.

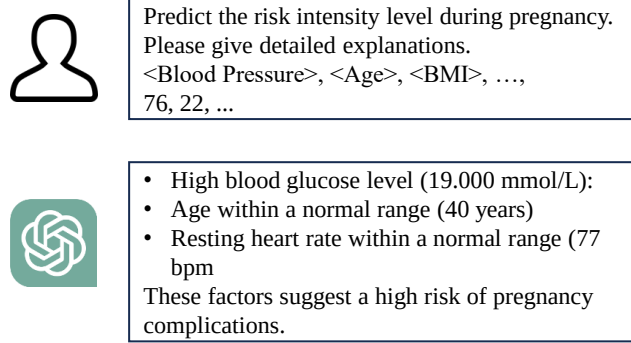


Figure 7: An example of LLM prediction with explanations.

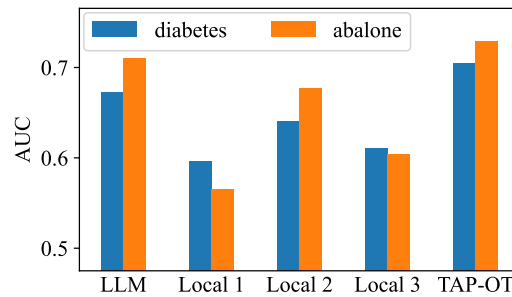


Figure 8: Application of TreePlugin in FL.

D.2 Application on Federated Learning

Federated Learning (FL) [32–34] is a popular machine learning paradigm wherein multiple parties collaboratively train a machine learning model without exchanging their local data, offering privacy advantages. Fine-tuning a public LLM with data from various parties within a federated framework presents significant challenges such as expensive computation costs and easily to be overfitted.

Our methodology offers a natural solution for parameter-efficient federated tuning of LLMs on tabular data. Rather than fine-tuning the LLM directly, each participating entity can locally employ our approach to train a decision tree. Subsequently, these trees can be amalgamated into an ensemble, facilitating the consolidation of knowledge across parties. During inference, the prompts from all trees are appended in our model. To emulate a vertical FL scenario, we partition *diabetes* and *abalone* [35] into three equal, random subsets along the feature dimension. We then compare the efficacy of using only the LLM for predictions, solely relying on locally trained trees, and employing our integrated approach, as illustrated in Figure 8. The results indicate that our method significantly enhances performance by harnessing the collective knowledge of multiple parties.

D.3 Results on Llama-2 70b

Besides GPT-3.5, we also evaluate the results of TreePlugin on Llama-2 70b. The respective results are tabulated in Table 8. The table reveals that our methodology consistently enhances the performance of each LLM, invariably surpassing the other baselines.

Table 8: Application of our approach with Llama-2 70b on diabetes (#shots = 8).

SP	SP-Tree	CoT-LLM	CoT-Tree	TreePlugin
0.72±0.02	0.65±0.03	0.72±0.04	0.74±0.03	0.76±0.02

Table 9: Overhead of TreePlugin in a single run on diabetes with default parameters (#shots = 2, $\lambda = 1$, $\mu = 0.1$).

	TreePlugin			TreePlugin-FS		
	computation (s)	#query	AUC	computation (s)	#query	AUC
diabetes	0.5	67	0.71	0.4	20	0.71
car	0.2	60	0.83	0.1	21	0.61
blood	0.4	35	0.74	0.3	14	0.69
maternal	0.4	49	0.73	0.2	20	0.72

429 D.4 Overhead of TreePlugin

430 TreePlugin requires multiple queries when training a tree. One way to reduce the number of queries
 431 is to directly ask the LLM to select an appropriate feature as the split feature instead of traversing all
 432 possible features when updating each tree node. We name this version of TreePlugin as TreePlugin-FS.
 433 We report the overhead and AUC of a single run in Table 9. We observe that the computation time on
 434 the user side is very small, indicating that our approach has little overhead for the user’s computing
 435 machine. The query time is the main bottleneck of TreePlugin. TreePlugin-FS can significantly
 436 reduce the number of queries. However, the performance of TreePlugin-FS is much worse than
 437 TreePlugin on the car dataset. In the future, we will exploit advanced ways to reduce the query
 438 overhead of TreePlugin while maintaining the prediction accuracy.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction have clearly summarized the paper's contributions and scope.

Guidelines:

- The answer NA means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: We have discussed the limitations of the work in Section 6.

Guidelines:

- The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory Assumptions and Proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [NA]

Justification: We do not present any theoretical result in the paper.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental Result Reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [\[Yes\]](#)

Justification: We have provided the details for reproducing the experiments in Section 5.1, Appendix A, and Appendix B.

Guidelines:

- The answer NA means that the paper does not include experiments.
- If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We have provided the code in the supplementary material.

Guidelines:

- The answer NA means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://nips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so “No” is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://nips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental Setting/Details

Question: Does the paper specify all the training and test details (e.g., data splits, hyper-parameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

Answer: [Yes]

Justification: We have provided the details in Section 5.1 and the code.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment Statistical Significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: We have reported the standard derivation in the experimental results.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.

- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments Compute Resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: We have provided the information in Section 5.1 and Appendix D.4.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code Of Ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: Our research has followed the NeurIPS Code of Ethics.

Guidelines:

- The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader Impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: We have discussed the potential societal impacts of the work performed in Section 6.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

Answer: [NA]

Justification: We do not release any new data or models.

Guidelines:

- The answer NA means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We have included the citations to the datasets in Section 5.1.

Guidelines:

- The answer NA means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.

- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New Assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: We have included the documentation for our code in the supplementary material.

Guidelines:

- The answer NA means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and Research with Human Subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [NA]

Justification: We do not involve crowdsourcing nor research with human subjects.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional Review Board (IRB) Approvals or Equivalent for Research with Human Subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [NA]

Justification: We do not involve crowdsourcing nor research with human subjects.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.